PROGRAMACIÓN AVANZADA · UTU · BACHILLERATO TECNOLÓGICO

Actividades de práctica — CRUD con Base de Datos

Tres actividades para completar el CRUD sobre MySQL con JDBC: agregar, buscar por código, actualizar y eliminar, aplicando las 5 fases del acceso a datos. Primero sobre la tabla personas, que incorpora un atributo nuevo, y después sobre la entidad propia.

TEMA 3 — JDBC + MYSQL | INSERT · UPDATE · DELETE | ACT. 03 — ENTIDAD PROPIA | DIFICULTAD CRECIENTE

CONTEXTO

La dirección del instituto decidió registrar también el **sueldo** de cada persona, y el sistema tiene que permitir cargarlo, verlo y modificarlo desde la aplicación. Hasta ahora el DAO solo consultaba la base (`SELECT`) y el resto de los métodos trabajaba con un `ArrayList` en memoria. En esta práctica se agrega el atributo nuevo en todas las capas y se completan los métodos del DAO para que **todas** las operaciones del CRUD se hagan sobre MySQL.

01

CONSOLIDACIÓN · TEMA 3 — INSERT

Agregar el sueldo: alta y listado desde la BD

NIVEL BÁSICO

Agregar a la tabla **personas** la columna **sueldo**, mostrarla en el listado y permitir dar de alta personas con su sueldo **desde el formulario de la aplicación**. Para eso, los métodos `agregar()` y `obtenerXCodigo()` del DAO dejan de usar el `ArrayList` y pasan a trabajar con MySQL.

PROYECTO BASE	SQL NUEVO
`mini-crud-sql-base`	**ALTER TABLE · INSERT**

// QUÉ CAMBIA EN CADA CAPA

CAPA	ARCHIVO	CAMBIO
Base de datos	`tabla personas`	Columna `sueldo` con `ALTER TABLE` y sueldos cargados con `UPDATE`
Modelo · VO	`PersonaVO.java`	Atributo `sueldo`, constructor de 3 parámetros, getter y setter
Modelo · DAO	`PersonaDAO.java`	`getLista()` lee el sueldo; `obtenerXCodigo()` y `agregar()` pasan a MySQL
Controlador	`PersonaControlador-Servlet.java`	En `agregar()`, leer el parámetro `sueldo`
Vista	`persona-form.jsp` `persona-lista.jsp`	Campo Sueldo en el formulario; columna Sueldo y botón Agregar habilitado en el listado

// TAREAS

1. En **MySQL Workbench**, agregar la columna, cargar el sueldo de las personas que ya están en la tabla y comprobar el resultado.

MySQL Workbench — columna nueva y datos

```
USE institutoWeb;

-- 1. Agregar la columna sin perder los datos que ya existen
ALTER TABLE personas ADD COLUMN sueldo DECIMAL(10,2);

-- 2. Cargar el sueldo de las personas que ya están en la tabla
UPDATE personas SET sueldo = 45000.00 WHERE codigo = 1;
UPDATE personas SET sueldo = 52300.50 WHERE codigo = 2;
UPDATE personas SET sueldo = 38900.00 WHERE codigo = 3;

-- 3. Comprobar el resultado
SELECT * FROM personas;
```

Si en la tabla quedaron otras personas de la Práctica 3, cargarles también un sueldo con un `UPDATE` por código. La columna `fecha_nacimiento`, si existe, no molesta: el proyecto simplemente no la usa.

2. En **PersonaVO.java**: declarar `private double sueldo`. Agregar un constructor de 3 parámetros (código, nombre y sueldo) sin borrar el de 2. Agregar getter y setter.

3. En **PersonaDAO.java**, método `getLista()`: en la fase 5 leer la columna con `rs.getDouble("sueldo")` y usar el constructor de 3 parámetros.

4. En **PersonaDAO.java**, método `obtenerXCodigo(int cod)`: reemplazar el recorrido del `ArrayList` por las 5 fases con `SELECT * FROM personas WHERE codigo = ?`. Como el parámetro se asigna antes de ejecutar, el `ResultSet` se abre en un try anidado. En la fase 5 se usa `if (rs.next())` y no `while`, porque el código es clave primaria: hay una fila o ninguna.

5. En **PersonaDAO.java**, método `agregar(PersonaVO p)`: reemplazar el `personasMemoria.add(p)` por un `INSERT` con tres parámetros. Se ejecuta con `executeUpdate()` y no tiene fase 5, porque no devuelve filas.

6. En **PersonaControladorServlet.java**, método privado `agregar()`: leer el sueldo con `Double.parseDouble(request.getParameter("sueldo"))` y crear la persona con el constructor de 3 parámetros.

7. En **persona-form.jsp**: agregar la etiqueta y el campo **Sueldo** (`type="number"`, `name="sueldo"`). En **persona-lista.jsp**: agregar `<th>Sueldo</th>` y la celda `${p.sueldo}`, cambiar el `colspan` de 3 a 4 y reemplazar el botón deshabilitado **Agregar Persona** por el enlace activo que ya está comentado debajo.

// REFERENCIAS DE CÓDIGO

PersonaVO.java — atributo y constructor nuevos

```java
private double sueldo;                                  // ← nuevo atributo

// Se mantiene el constructor de 2 parámetros
public PersonaVO(int cod, String nombre) { ... }

// Constructor nuevo, con el sueldo (3 parámetros)
public PersonaVO(int cod, String nombre, double sueldo) {
    this.cod = cod;
    this.nombre = nombre;
    this.sueldo = sueldo;
}

public double getSueldo() {
    return sueldo;
}

public void setSueldo(double sueldo) {
    this.sueldo = sueldo;
}
```

PersonaDAO.java — agregar() con JDBC

```java
public String agregar(PersonaVO p) {
    // FASE 1: Definir la sentencia SQL
    String sql = "INSERT INTO personas(codigo, nombre, sueldo) VALUES (?, ?, ?)";
    String mensaje = "Error al agregar persona.";

    // FASES 2 y 3: conexión y PreparedStatement en try-with-resources
    try (Connection con = Conexion.obtener();
         PreparedStatement ps = con.prepareStatement(sql)) {

        // FASE 3: Asignar parámetros (uno por cada ?)
        ps.setInt(1, p.getCodigo());
        ps.setString(2, p.getNombre());
        ps.setDouble(3, p.getSueldo());                    // ← nuevo
        // FASE 4: Ejecutar (INSERT no tiene FASE 5)
        int filasAfectadas = ps.executeUpdate();
        if (filasAfectadas > 0) mensaje = "Persona agregada.";
    } catch (SQLException e) {
        System.err.println("Error al agregar persona: " + e.getMessage());
    }
    return mensaje;
}
```

PersonaDAO.java — estructura de obtenerXCodigo()

```java
public PersonaVO obtenerXCodigo(int cod) {
    // FASE 1: Definir la sentencia SQL
    String sql = "SELECT * FROM personas WHERE codigo = ?";
    PersonaVO persona = null;

    // FASES 2 y 3: conexión y PreparedStatement
    try (Connection con = Conexion.obtener();
         PreparedStatement ps = con.prepareStatement(sql)) {

        // FASE 3: asignar el parámetro
        ps.setInt(1, cod);
        // FASE 4: Ejecutar (ResultSet en try anidado)
        try (ResultSet rs = ps.executeQuery()) {
            // FASE 5: if, porque codigo es clave primaria
            if (rs.next()) {
                persona = new PersonaVO( ... );             // código, nombre y sueldo
            }
        }
    } catch (SQLException e) {
        System.err.println("Error al obtener persona: " + e.getMessage());
    }
    return persona;
}
```

persona-form.jsp — campo nuevo

```jsp
<label>Sueldo</label>
<input type="number" name="sueldo" step="0.01" min="0" required
       value="${requestScope.persona.sueldo}">
```

¿POR QUÉ TAMBIÉN OBTENERXCODIGO()?

Antes de agregar, el Servlet llama a `dao.obtenerXCodigo(codigo)` para comprobar que el código no esté repetido. Si ese método siguiera buscando en el `ArrayList`, nunca encontraría a las personas guardadas en MySQL: la comprobación no detectaría el código duplicado y el `INSERT` fallaría por clave primaria repetida, sin ningún mensaje para el usuario.

// CRITERIOS DE VERIFICACIÓN

- [ ] En MySQL, `SELECT * FROM personas` muestra la columna `sueldo` con los valores cargados.
- [ ] El listado de la aplicación muestra la columna Sueldo con los mismos valores que MySQL.
- [ ] Al agregar una persona desde el formulario, aparece en el listado y también en el `SELECT` de Workbench.
- [ ] Al intentar agregar un código que ya existe, el formulario muestra el mensaje de error y no se inserta nada.
- [ ] Después de detener y volver a iniciar Tomcat, la persona agregada sigue en el listado.

02

APLICACIÓN · TEMA 3 — UPDATE Y DELETE

Editar y eliminar personas en la BD

NIVEL INTERMEDIO

Completar el CRUD: los métodos `actualizar()` y `eliminar()` del DAO pasan a MySQL, el formulario de edición incorpora el sueldo y se habilitan los botones **Editar** y **Eliminar**. Al final, el `ArrayList` en memoria desaparece del DAO: todas las operaciones se hacen sobre la base.

PROYECTO BASE	SQL NUEVO
el de la Actividad 01	UPDATE · DELETE

// CONCEPTO: EXECUTEUPDATE()

SENTENCIA	MÉTODO DE EJECUCIÓN	QUÉ DEVUELVE
SELECT	executeQuery()	Un `ResultSet` con las filas encontradas (se lee en la fase 5)
INSERT · UPDATE · DELETE	executeUpdate()	Un `int` con la cantidad de filas afectadas: 0 si el código no existía

// TAREAS

1. En **PersonaDAO.java**, método `actualizar(PersonaVO p)`: aplicar las fases con `UPDATE personas SET nombre = ?, sueldo = ? WHERE codigo = ?`. Los parámetros se asignan en el orden de los `?`: el código es el tercero. Si `executeUpdate()` devuelve más de 0, el mensaje es "Persona actualizada.".

2. En **PersonaDAO.java**, método `eliminar(int codigo)`: aplicar las fases con `DELETE FROM personas WHERE codigo = ?`, siguiendo el mismo esquema que `actualizar()`.

3. En **PersonaControladorServlet.java**, método privado `editar()`: leer el sueldo con `Double.parseDouble(...)` y llamar a `dao.actualizar()` con el constructor de 3 parámetros. `eliminar()` y `cargarEditar()` no cambian.

4. En **persona-editar.jsp**: agregar el campo **Sueldo** con `value="${requestScope.persona.sueldo}"`, para que el formulario aparezca con el valor actual.

5. En **persona-lista.jsp**: reemplazar los botones deshabilitados **Editar** y **Eliminar** de cada fila por el enlace y el formulario activos que ya están comentados en el mismo archivo. Eliminar sigue enviándose por `POST`.

6. En **PersonaDAO.java**: borrar el atributo `personasMemoria`, el constructor que lo crea y los métodos que ya no se usan (`estaPersona()`, `devolver()` y `cantidad()`). Compilar con `mvn clean package` para comprobar que nada dependía de ellos.

// REFERENCIAS DE CÓDIGO

PersonaDAO.java — actualizar() con JDBC

```java
public String actualizar(PersonaVO p) {
    // FASE 1: Definir la sentencia SQL
    String sql = "UPDATE personas SET nombre = ?, sueldo = ? WHERE codigo = ?";
    String mensaje = "Persona no encontrada.";

    // FASES 2 y 3: conexión y PreparedStatement en try-with-resources
    try (Connection con = Conexion.obtener();
         PreparedStatement ps = con.prepareStatement(sql)) {

        // FASE 3: Asignar parámetros, en el orden de los ? del SQL
        ps.setString(1, p.getNombre());
        ps.setDouble(2, p.getSueldo());
        ps.setInt(3, p.getCodigo());                        // el WHERE va último
        // FASE 4: Ejecutar (devuelve cuántas filas cambiaron)
        int filasAfectadas = ps.executeUpdate();
        if (filasAfectadas > 0) mensaje = "Persona actualizada.";
    } catch (SQLException e) {
        System.err.println("Error al actualizar persona: " + e.getMessage());
        mensaje = "Error al actualizar persona.";
    }
    return mensaje;
}
```

PersonaControladorServlet.java — editar()

```java
private void editar(HttpServletRequest request, HttpServletResponse response)
        throws IOException {
    int codigo = Integer.parseInt(request.getParameter("codigo"));
    String nombre = request.getParameter("nombre");
    double sueldo = Double.parseDouble(request.getParameter("sueldo"));    // ← nuevo
    dao.actualizar(new PersonaVO(codigo, nombre, sueldo));
    // PRG → volver a listar
    response.sendRedirect(request.getContextPath() + "/persona");
}
```

persona-editar.jsp — campo nuevo

```jsp
<label>Sueldo</label>
<input type="number" name="sueldo" step="0.01" min="0"
       value="${requestScope.persona.sueldo}" required>
```

EL ORDEN DE LOS PARÁMETROS IMPORTA

El número de `ps.setXxx(n, ...)` indica a qué `?` del SQL corresponde el valor, contando de izquierda a derecha. Si el código se asignara en la posición 1, el `UPDATE` intentaría guardar un número en `nombre` y buscaría por código un texto: no se actualizaría ninguna fila y no aparecería ningún error.

// CRITERIOS DE VERIFICACIÓN

- [ ] Al editar el sueldo de una persona, el listado muestra el valor nuevo y `SELECT * FROM personas` en Workbench también.
- [ ] En el formulario de edición el código no se puede modificar y los campos aparecen con los valores actuales.
- [ ] Al eliminar una persona (después de confirmar), desaparece del listado y de la tabla en MySQL.
- [ ] `PersonaDAO` ya no tiene el `ArrayList`: todos sus métodos usan `Conexion.obtener()`.
- [ ] Después de reiniciar Tomcat, los cambios hechos desde la aplicación se mantienen.

03

DESAFÍO · TEMA 3 — DE MEMORIA A MYSQL (2ª PARTE)

CRUD completo de la entidad propia en MySQL

NIVEL AVANZADO

Retomar el proyecto de la **Actividad 03 de la Práctica 3**, donde el listado de la entidad propia ya sale de MySQL y los botones están deshabilitados. Igual que con el sueldo de las personas, agregar a la entidad un **atributo numérico nuevo** y completar en el DAO las operaciones de agregar, buscar por código, actualizar y eliminar con JDBC. Los ejemplos usan **Producto** con el atributo nuevo **stock**: reemplazar por la entidad propia.

PROYECTO BASE	REFERENCIA
Act. 03 · Práctica 3	**PersonaDAO de las Act. 01 y 02**

// IDEAS DE ATRIBUTO NUEVO

ENTIDAD	ATRIBUTO (JAVA)	COLUMNA (MYSQL)
Producto	`int stock`	`stock INT`
Libro	`int cantidadPaginas`	`cantidad_paginas INT`
Película	`int duracionMinutos`	`duracion_minutos INT`
Videojuego	`double precio`	`precio DECIMAL(8,2)`
Estudiante	`double promedio`	`promedio DECIMAL(4,2)`

Si la entidad ya tiene alguno de estos atributos, elegir otro que tenga sentido y acordarlo con la docente.

// TAREAS

1. En **MySQL Workbench**: agregar la columna con `ALTER TABLE` y cargar valores en los registros existentes con `UPDATE`. Agregar estas sentencias al final del script `entidad.sql` de la Práctica 3.

MySQL Workbench — columna nueva (ejemplo con Producto)

```
USE institutoWeb;

ALTER TABLE productos ADD COLUMN stock INT;

UPDATE productos SET stock = 25 WHERE codigo = 1;
UPDATE productos SET stock = 40 WHERE codigo = 2;
UPDATE productos SET stock = 8  WHERE codigo = 3;
UPDATE productos SET stock = 15 WHERE codigo = 4;

SELECT * FROM productos;
```

2. En el **VO**: agregar el atributo, un constructor que lo incluya, y su getter y setter.

3. En el **DAO**: `getLista()` lee la columna nueva; `obtenerXCodigo()`, `agregar()`, `actualizar()` y `eliminar()` se reescriben con las 5 fases, tomando como modelo los de `PersonaDAO`. Borrar el `ArrayList` en memoria y los métodos que ya no se usen. Los nombres y las firmas de los métodos se mantienen, así el Servlet sigue llamándolos igual.

4 En el **Servlet**: leer el atributo nuevo en `agregar()` y en `editar()`, con `Integer.parseInt()` o `Double.parseDouble()` según el tipo. Antes de agregar, comprobar con `obtenerXCodigo()` que el código no exista.

5 En las **vistas**: columna nueva en el listado, campo nuevo en los formularios de alta y edición, y los botones Agregar, Editar y Eliminar habilitados otra vez.

// REFERENCIAS DE CÓDIGO

ProductoDAO.java — SQL de cada método (ejemplo)

```
// obtenerXCodigo(int cod)
String sql = "SELECT * FROM productos WHERE codigo = ?";

// agregar(ProductoVO p): 5 parámetros, en el orden de las columnas
String sql = "INSERT INTO productos(codigo, nombre, precio, descripcion, stock) "
           + "VALUES (?, ?, ?, ?, ?)";

// actualizar(ProductoVO p): el código va último, en el WHERE
String sql = "UPDATE productos SET nombre = ?, precio = ?, descripcion = ?, stock = ? "
           + "WHERE codigo = ?";

// eliminar(int codigo)
String sql = "DELETE FROM productos WHERE codigo = ?";
```

// ARCHIVOS A CREAR Y MODIFICAR

Estructura esperada (ejemplo con Producto)

```
Base de datos:
  tabla productos   ← ALTER TABLE + UPDATE (en entidad.sql)

Capa Modelo:
  src/main/java/modelo/vo/ProductoVO.java    ← atributo stock
  src/main/java/modelo/dao/ProductoDAO.java  ← CRUD completo con JDBC

Capa Controlador:
  src/main/java/controlador/ProductoControladorServlet.java  ← lee stock

Capa Vista:
  src/main/webapp/WEB-INF/vista/producto-lista.jsp    ← columna + botones activos
  src/main/webapp/WEB-INF/vista/producto-form.jsp     ← campo stock
  src/main/webapp/WEB-INF/vista/producto-editar.jsp   ← campo stock
```

QUÉ CAMBIA Y QUÉ NO

Como los métodos del DAO conservan su nombre y lo que reciben y devuelven, el Servlet solo cambia por el atributo nuevo, no por pasar a MySQL. Esa es la ventaja de que el acceso a datos esté separado en su propia capa.

// CRITERIOS DE VERIFICACIÓN

- [ ] El listado muestra la columna nueva con los mismos valores que el `SELECT` en Workbench.
- [ ] Agregar, editar y eliminar desde la aplicación se reflejan en la tabla de MySQL.
- [ ] Al agregar un código que ya existe se muestra un mensaje de error y no se inserta nada.
- [ ] El DAO ya no tiene `ArrayList` en memoria y los datos se mantienen al reiniciar Tomcat.
- [ ] La entrega incluye `entidad.sql` actualizado, con el `ALTER TABLE` y los `UPDATE`.